LAPORAN PRAKTIKUM
Modul 8: HMVC, Otorisasi, dan AJAX



Disusun Oleh:

Nama : Davin Aileenskie Sofiandy

NIM : 707012500135

Kelas : D4SIKC-49-04

Mata Kuliah : Pengembangan Aplikasi Berbasis Web

TELKOM UNIVERSITY

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan Praktikum

Praktikum Modul 8 ini saya kerjakan pada project PRK_TA_8, yaitu aplikasi perpustakaan SiPustaka yang dibangun sebagai tugas akhir praktikum. Isinya katalog buku, pencatatan peminjaman, dan pengelolaan akun pengguna. Materi HMVC, otorisasi berbasis role, dan AJAX masih sama seperti modul, tetapi yang saya terapkan kali ini adalah seluruh aplikasi, bukan satu modul uji coba. Semua fitur tersebut saya letakkan di dalam modul terpisah supaya kode bisa dibaca per bagian.

Tujuan yang ingin saya capai:

1. Memecah fitur aplikasi perpustakaan menjadi modul HMVC dengan paket nwidart/laravel-modules.

2. Membatasi akses berdasarkan role admin dan user lewat satu middleware yang dipakai berulang di beberapa modul.

3. Membuat fitur register dan login manual di luar folder Modules, dengan akun baru otomatis berrole user.

4. Membangun modul UserManagement dengan CRUD akun sinkron yang hanya bisa dibuka admin.

5. Membangun modul Buku dengan CRUD sinkron dan pencarian judul, penulis, atau penerbit.

6. Membangun modul Peminjaman dengan CRUD AJAX, termasuk unggah bukti peminjaman berformat gambar atau PDF.

7. Memakai Form Request untuk validasi dan menampilkan pesan error per input, baik pada form biasa maupun form AJAX.

2. Alat dan Bahan

Praktikum saya jalankan di laptop pribadi dengan Windows dan XAMPP sebagai server lokal. Seluruh versi di bawah ini saya ambil langsung dari perintah di terminal laptop, bukan dari contoh di modul. Bootstrap, Bootstrap Icons, dan jQuery dimuat lewat CDN, jadi project ini tidak memerlukan Node.js maupun proses build aset.

2.1 Perangkat Lunak

XAMPP	Apache dan MySQL/MariaDB, terpasang di D:\xampp
MySQL / MariaDB	10.4.32, database db_sipustaka
PHP	8.2.12 (ZTS Visual C++ 2019 x64)
Composer	2.10.3
Laravel	12.71.0

Framework	
nwidart/laravel-modules	v12.0.5, beserta wikimedia/composer-merge-plugin v2.1.0
Pest	v3.8.7
Bootstrap 5.3, Bootstrap Icons, jQuery 3.6	Dimuat dari CDN lewat layout, tanpa Node.js
Visual Studio Code	Editor kode dan terminal
Google Chrome	Menguji tampilan aplikasi

Semua versi di tabel saya ambil dari perintah php -v, composer -V, php artisan --version, dan composer.lock di laptop. Database bernama db_sipustaka dengan konfigurasi DB_CONNECTION=mysql, DB_HOST=127.0.0.1, DB_PORT=3306, dan DB_USERNAME=root tanpa password.

3. Langkah-Langkah Praktikum

Saya mulai dari memastikan paket modul dan konfigurasi dasarnya sudah benar, lalu masuk ke bagian otorisasi. Setelah middleware role berjalan, tiga modul dibangun satu per satu: UserManagement yang sinkron, Buku yang sinkron dengan pencarian, dan Peminjaman yang seluruh operasinya memakai AJAX. Bagian terakhir menghubungkan semuanya lewat dashboard dan navigasi.

3.1 Memasang Paket Modul dan Mengatur Composer

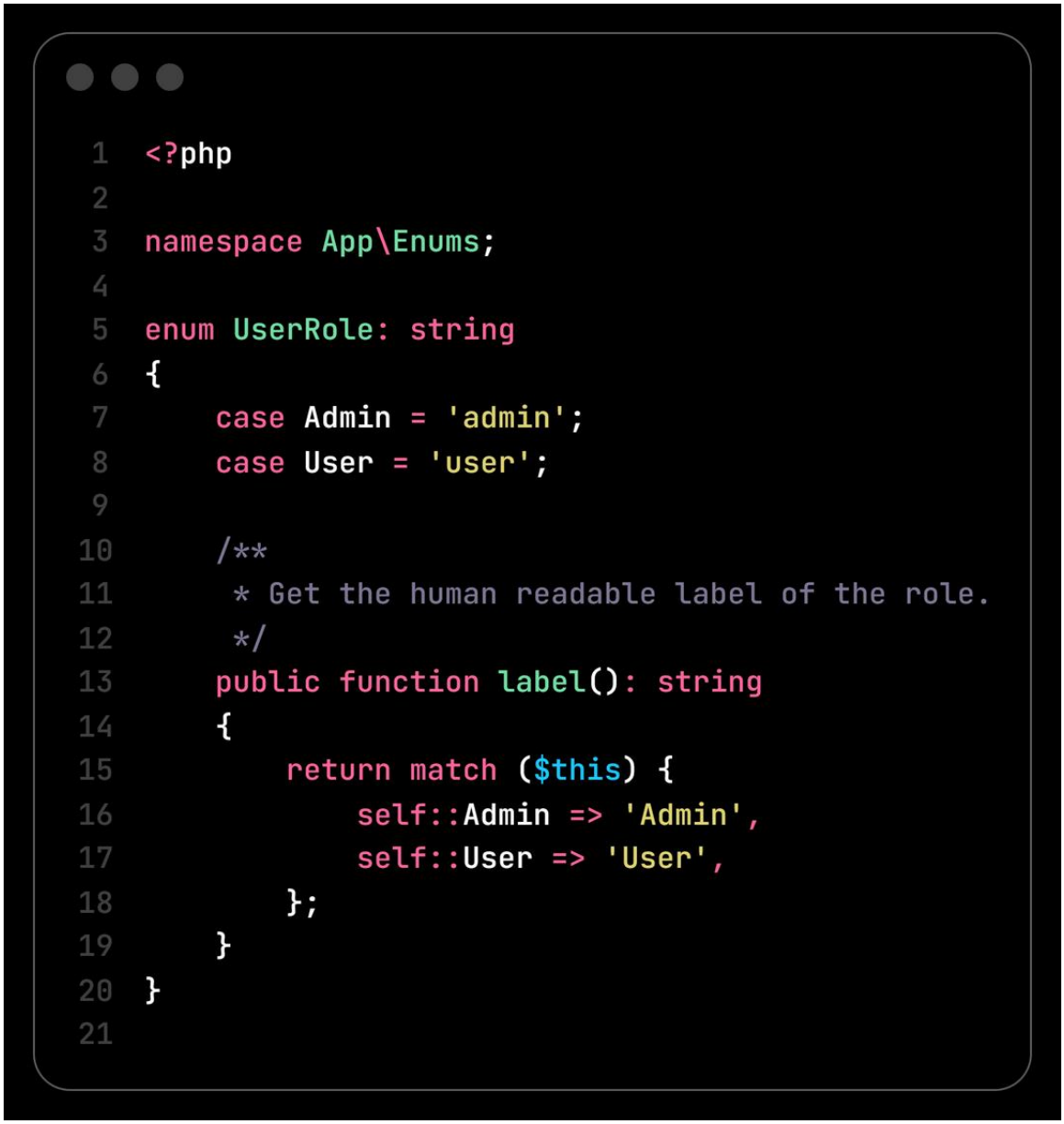
Aplikasi dasar dibuat dari skeleton Laravel 12, kemudian paket modul ditambahkan dengan composer require nwidart/laravel-modules. Pada composer.json saya menambahkan blok extra.merge-plugin yang mengarah ke Modules/*/composer.json. Blok itu membuat Composer ikut membaca dependency milik setiap modul, sehingga modul baru tidak perlu pengaturan autoload terpisah. Saya juga menambahkan allow-plugins untuk wikimedia/composer-merge-plugin agar Composer tidak meminta konfirmasi interaktif saat install.

```
{
    "$schema": "https:// getcomposer.org/schema.json",
    "name": "laravel/laravel",
    "type": "project",
    "description": "The skeleton application for the Laravel framework.",
    "keywords": ["laravel", "framework"],
    "license": "MIT",
    "require": {
        "php": "^8.2",
        "laravel/framework": "^12.0",
        "laravel/tinker": "^2.10.1",
        "nwidart/laravel-modules": "^12.0"
    },
    "require-dev": {
        "fakerphp/faker": "^1.23",
        "laravel/pail": "^1.2.2",
        "laravel/pint": "^1.24",
        "laravel/sail": "^1.41",
        "mockery/mockery": "^1.6",
        "nunomaduro/collision": "^8.6",
        "pestphp/pest": "^3.8",
        "pestphp/pest-plugin-laravel": "^3.2",
        "phpunit/phpunit": "^11.5.50"
    },
    "autoload": {
        "psr-4": {
            "App\\": "app/",
            "Database\\Factories\\": "database/factories/",
            "Database\\Seeders\\": "database/seeders/"
        }
    },
    "autoload-dev": {
        "psr-4": {
            "Tests\\": "tests/"
        }
    },
    "scripts": {
        "setup": [
            "composer install",
            "@php -r \"file_exists('.env') || copy('.env.example', '.env');\"",
            "@php artisan key:generate",
            "@php artisan migrate --force",
            "npm install",
            "npm run build"
        ],
        "dev": [
            "Composer\\Config::disableProcessTimeout",
            "npx concurrently -c \"#93c5fd,#c4b5fd,#fb7185,#fdba74\" \"php artisan serve\" \"php artisan queue:listen --tries=1 --timeout=0\" \"php artisan pail --timeout=0\" \"npm run dev\" --names=server,queue,logs,vite --kill-others"
        ],
        "test": [
            "@php artisan config:clear --ansi",
            "@php artisan test"
        ],
        "post-autoload-dump": [
            "Illuminate\\Foundation\\ComposerScripts::postAutoloadDump",
            "@php artisan package:discover --ansi"
        ],
        "post-update-cmd": [
            "@php artisan vendor:publish --tag=laravel-assets --ansi --force"
        ],
        "post-root-package-install": [
            "@php -r \"file_exists('.env') || copy('.env.example', '.env');\""
        ],
```

3.2 Menambahkan Kolom Role dan Enum UserRole

Tabel users dari migration bawaan Laravel belum punya kolom role. Saya menambahkannya lewat migration baru, bukan mengubah migration awal, dengan string('role')->default('user')->after('email'). Nilai bawaan user dipakai supaya akun yang mendaftar sendiri otomatis menjadi anggota biasa.

Nilai admin dan user saya bungkus dalam enum UserRole di app/Enums/UserRole.php. Enum ini punya method label() untuk menampilkan nama role di halaman. Dengan enum, nilai role yang salah akan ditolak oleh aturan Rule::enum(UserRole::class) sebelum sampai ke database.



```php
<?php

namespace App\Enums;

enum UserRole: string
{
    case Admin = 'admin';
    case User = 'user';

    /**
     * Get the human readable label of the role.
     */
    public function label(): string
    {
        return match ($this) {
            self::Admin => 'Admin',
            self::User => 'User',
        };
    }
}

```

3.3 Model User dan Middleware Role

Model User diperbarui dengan menambah role ke $fillable, memberi nilai bawaan user, dan melakukan cast ke UserRole. Dua method tambahan dipakai di banyak tempat. isAdmin() dipakai untuk menyembunyikan menu dan menghitung data di dashboard, sedangkan hasRole() menerima beberapa role sekaligus dan dipanggil dari middleware.

```php
<?php

namespace App\Models;

// use Illuminate\Contracts\Auth\MustVerifyEmail;
use App\Enums\UserRole;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'name',
        'email',
        'role',
        'password',
    ];

    /**
     * The attributes that should be hidden for serialization.
     *
     * @var list<string>
     */
    protected $hidden = [
        'password',
        'remember_token',
    ];

    /**
     * The model's default attribute values.
     *
     * @var array<string, mixed>
     */
    protected $attributes = [
        'role' => 'user',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'role' => UserRole::class,
        ];
    }

    /**
     * Determine whether the user has the admin role.
     */
    public function isAdmin(): bool
```

Middleware EnsureUserHasRole mengambil pengguna yang sedang login. Kalau pengguna tidak ada atau role-nya tidak termasuk daftar yang diminta, request dihentikan dengan abort(403) dan pesan berbahasa Indonesia. Parameter seperti role:admin,user diterima sebagai variadic string, sehingga satu grup route bisa dibuka dua role sekaligus.

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
    /**
     * Ensure the authenticated user has one of the given roles.
     *
     * Usage: ->middleware('role:admin') or ->middleware('role:admin,user')
     *
     * @param  Closure(Request): (Response)  $next
     */
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        $user = $request->user();

        if ($user === null || ! $user->hasRole(...$roles)) {
            abort(403, 'Anda tidak memiliki hak akses ke halaman ini.');
        }

        return $next($request);
    }
}

```

Alias role didaftarkan pada bootstrap/app.php. File yang sama juga mengatur tujuan redirect: tamu diarahkan ke halaman login dan pengguna yang sudah login diarahkan ke dashboard.

```php
<?php

use App\Http\Middleware\EnsureUserHasRole;
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->alias([
            'role' => EnsureUserHasRole::class,
        ]);

        $middleware->redirectGuestsTo(fn () => route('login'));
        $middleware->redirectUsersTo(fn () => route('dashboard'));
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();

```

3.4 Seeder dan Registrasi Mandiri

DatabaseSeeder membuat dua akun uji, yaitu Administrator dengan email admin@example.com berrole admin dan Anggota Perpustakaan dengan email user@example.com berrole user. Keduanya memakai password password. Setelah itu seeder modul Buku dan Peminjaman dipanggil agar tabel langsung berisi data contoh.

```php
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        User::factory()->admin()->create([
            'name' => 'Administrator',
            'email' => 'admin@example.com',
            'password' => 'password',
        ]);

        User::factory()->create([
            'name' => 'Anggota Perpustakaan',
            'email' => 'user@example.com',
            'password' => 'password',
        ]);

        $this->call([
            \Modules\Buku\Database\Seeders\BukuDatabaseSeeder::class,
            \Modules\Peminjaman\Database\Seeders\PeminjamanDatabaseSeeder::class,
        ]);
    }
}

```

Halaman register berada di luar folder Modules dan dijaga middleware guest. Email dan password divalidasi oleh RegisterRequest yang tidak memiliki field role. Di controller, role diisi langsung dengan UserRole::User, lalu pengguna langsung diarahkan ke dashboard dengan pesan registrasi berhasil.

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Enums\UserRole;
use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\RegisterRequest;
use App\Models\User;
use Illuminate\Http\RedirectResponse;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\DB;
use Illuminate\View\View;

class RegisterController extends Controller
{
    /**
     * Show the registration form.
     */
    public function create(): View
    {
        return view('auth.register');
    }

    /**
     * Register a new user. Self-registered accounts always get the
     * "user" role.
     */
    public function store(RegisterRequest $request): RedirectResponse
    {
        $user = DB::transaction(function () use ($request) {
            return User::create([
                ...$request->safe()->only(['name', 'email', 'password']),
                'role' => UserRole::User,
            ]);
        });

        Auth::login($user);

        $request->session()->regenerate();

        return redirect()->route('dashboard')->with('success', 'Registrasi berhasil.
Selamat datang!');
    }
}

```

3.5 Modul Buku: Migration, Model, dan Controller

Modul Buku saya buat dengan php artisan module:make Buku. Tabel bukus berisi judul, penulis, penerbit, tahun_terbit bertipe year, dan stok sebagai unsigned integer dengan nilai bawaan 0. Model Buku meng-cast tahun dan stok ke integer serta menyediakan method tersedia() yang mengembalikan true selama stok lebih dari nol.

```php
<?php

namespace Modules\Buku\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Modules\Buku\Database\Factories\BukuFactory;

class Buku extends Model
{
    /** @use HasFactory<BukuFactory> */
    use HasFactory;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'judul',
        'penulis',
        'penerbit',
        'tahun_terbit',
        'stok',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'tahun_terbit' => 'integer',
            'stok' => 'integer',
        ];
    }

    /**
     * Determine whether the book is still available.
     */
```

BukuController membagi aksesnya ke dua tingkat. index dipakai bersama admin dan user, sedangkan create, store, edit, update, dan destroy hanya untuk admin. Method index membaca parameter search dan mencocokkannya ke judul, penulis, atau penerbit sekaligus, lalu mengurutkan hasil berdasarkan judul dengan paginate(10). withQueryString() menjaga kata kunci tetap ada saat pindah halaman.

```php
<?php

namespace Modules\Buku\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\View\View;
use Modules\Buku\Http\Requests\StoreBukuRequest;
use Modules\Buku\Http\Requests\UpdateBukuRequest;
use Modules\Buku\Models\Buku;

class BukuController extends Controller
{
    /**
     * Display a listing of books, optionally filtered by keyword.
     */
    public function index(Request $request): View
    {
        $search = $request->string('search')->trim()->toString();

        $bukus = Buku::query()
            ->when($search !== '', function (Builder $query) use ($search) {
                $query->where(function (Builder $query) use ($search) {
                    $query->where('judul', 'like', "%{$search}%")
                        ->orWhere('penulis', 'like', "%{$search}%")
                        ->orWhere('penerbit', 'like', "%{$search}%");
                });
            })
            ->orderBy('judul')
            ->paginate(10)
            ->withQueryString();

        return view('buku::index', [
            'bukus' => $bukus,
            'search' => $search,
        ]);
    }

    /**
     * Show the form for creating a new book.
     */
    public function create(): View
    {
        return view('buku::create');
    }

    /**
     * Store a newly created book.
     */
    public function store(StoreBukuRequest $request): RedirectResponse
    {
        Buku::create($request->validated());

        return redirect()->route('buku.index')
            ->with('success', 'Buku berhasil ditambahkan.');
    }

    /**
     * Show the form for editing the specified book.
```

Validasi store dan update buku dipisah ke StoreBukuRequest dan UpdateBukuRequest. Tahun terbit dibatasi minimal 1900 dan maksimal tahun berjalan, sedangkan stok tidak boleh negatif. Method authorize() pada request juga memeriksa isAdmin(), jadi request ini menolak pengiriman dari akun user meskipun route-nya dilewati.

```php
<?php

namespace Modules\Buku\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreBukuRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $tahunSekarang = (int) date('Y');

        return [
            'judul' => ['required', 'string', 'max:255'],
            'penulis' => ['required', 'string', 'max:255'],
            'penerbit' => ['required', 'string', 'max:255'],
            'tahun_terbit' => ['required', 'integer', 'min:1900', "max:{$tahunSekarang}"],

            'stok' => ['required', 'integer', 'min:0', 'max:10000'],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'judul.required' => 'Judul buku wajib diisi.',
            'penulis.required' => 'Nama penulis wajib diisi.',
            'penerbit.required' => 'Nama penerbit wajib diisi.',
            'tahun_terbit.required' => 'Tahun terbit wajib diisi.',
            'tahun_terbit.integer' => 'Tahun terbit harus berupa angka.',
            'tahun_terbit.min' => 'Tahun terbit minimal 1900.',
            'tahun_terbit.max' => 'Tahun terbit tidak boleh melebihi tahun sekarang.',
            'stok.required' => 'Stok wajib diisi.',
            'stok.integer' => 'Stok harus berupa angka.',
            'stok.min' => 'Stok tidak boleh bernilai negatif.',
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }
}

```

Route modul Buku dibagi dua grup. Grup pertama memakai middleware auth dan role:admin,user untuk satu route index. Grup kedua memakai auth dan role:admin untuk lima route perubahan data. Pembagian ini yang membuat tombol Tambah, Edit, dan Hapus cukup disembunyikan di view untuk pengguna biasa, sementara penolakan sebenarnya tetap terjadi di server.

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\Buku\Http\Controllers\BukuController;

// Katalog buku dapat dilihat oleh admin maupun user yang sudah login.
Route::middleware(['auth', 'role:admin,user'])->group(function () {
    Route::get('buku', [BukuController::class, 'index'])->name('buku.index');
});

// Perubahan data buku hanya boleh dilakukan admin.
Route::middleware(['auth', 'role:admin'])->group(function () {
    Route::get('buku/create', [BukuController::class, 'create'])->name('buku.create');
    Route::post('buku', [BukuController::class, 'store'])->name('buku.store');
    Route::get('buku/{buku}/edit', [BukuController::class, 'edit'])->name('buku.edit');
    Route::put('buku/{buku}', [BukuController::class, 'update'])->name('buku.update');
    Route::delete('buku/{buku}', [BukuController::class, 'destroy'])-
>name('buku.destroy');
});

```

3.6 Modul UserManagement

Modul UserManagement memakai CRUD sinkron yang hanya bisa dibuka admin. Route-nya memakai prefix admin dan awalan nama admin., dengan satu baris Route::resource('users')->except('show') di dalam middleware auth dan role:admin. Hasilnya index berada di /admin/users dengan nama route admin.users.index, dan halaman detail user tidak dibuat karena tidak diperlukan.

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\UserManagement\Http\Controllers\UserManagementController;

Route::middleware(['auth', 'role:admin'])
    ->prefix('admin')
    ->name('admin.')
    ->group(function () {
        Route::resource('users', UserManagementController::class)->except('show');
    });

```

UserManagementController menyediakan pencarian nama atau email, filter role, dan pagination sepuluh data. Kata kunci dicocokkan dengan like pada dua kolom, sedangkan filter role hanya aktif kalau nilainya valid sebagai UserRole. Saat menghapus, akun admin tidak boleh menghapus dirinya sendiri, sehingga controller mengembalikan pesan error dan tidak memanggil delete().

```php
<?php

namespace Modules\UserManagement\Http\Controllers;

use App\Enums\UserRole;
use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\View\View;
use Modules\UserManagement\Http\Requests\StoreUserRequest;
use Modules\UserManagement\Http\Requests\UpdateUserRequest;

class UserManagementController extends Controller
{
    /**
     * Display a listing of users, optionally filtered by keyword and role.
     */
    public function index(Request $request): View
    {
        $search = $request->string('search')->trim()->toString();
        $role = UserRole::tryFrom($request->string('role')->toString());

        $users = User::query()
            ->when($search !== '', function (Builder $query) use ($search) {
                $query->where(function (Builder $query) use ($search) {
                    $query->where('name', 'like', "%{$search}%")
                        ->orWhere('email', 'like', "%{$search}%");
                });
            })
            ->when($role, fn (Builder $query) => $query->where('role', $role))
            ->orderBy('name')
            ->paginate(10)
            ->withQueryString();

        return view('usermanagement::index', [
            'users' => $users,
            'search' => $search,
            'selectedRole' => $role,
            'roles' => UserRole::cases(),
        ]);
    }

    /**
     * Show the form for creating a new user.
     */
    public function create(): View
    {
        return view('usermanagement::create', ['roles' => UserRole::cases()]);
    }

    /**
     * Store a newly created user.
     */
    public function store(StoreUserRequest $request): RedirectResponse
    {
        User::create($request->validated());

        return redirect()->route('admin.users.index')
            ->with('success', 'User berhasil ditambahkan.');
    }
```

UpdateUserRequest memakai Rule::unique()->ignore() supaya email akun yang sedang diedit tidak dianggap duplikat. Ada aturan tambahan berupa Rule::when() untuk kasus admin mengedit akunnya sendiri: role hanya boleh tetap admin. Aturan itu mencegah admin menurunkan role-nya sendiri lalu terkunci dari halaman kelola user.

```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class UpdateUserRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * An admin cannot demote their own account, so they never lock
     * themselves out.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $user = $this->route('user');
        $isEditingOwnAccount = $user?->is($this->user()) ?? false;

        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => [
                'required',
                'string',
                'email',
                'max:255',
                Rule::unique('users', 'email')->ignore($user?->id),
            ],
            'role' => [
                'required',
                Rule::enum(UserRole::class),
                Rule::when(
                    $isEditingOwnAccount,
                    Rule::in([UserRole::Admin->value]),
                ),
            ],
            'password' => ['nullable', 'confirmed', Password::defaults()],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'role.in' => 'Anda tidak dapat menurunkan role akun Anda sendiri.',
        ];
    }

    /**
```

3.7 Modul Peminjaman: Migration dan Model

Modul Peminjaman mencatat transaksi pinjam. Migration pertama membuat tabel peminjamans berisi nama_peminjam, judul_buku, tanggal_pinjam, tanggal_kembali yang boleh kosong, dan status enum dengan dua nilai, Dipinjam dan Dikembalikan. Migration kedua menambahkan kolom bukti untuk path file dan nama_bukti untuk nama asli file yang diunggah.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('peminjamans', function (Blueprint $table) {
            $table->id();
            $table->string('nama_peminjam');
            $table->string('judul_buku');
            $table->date('tanggal_pinjam');
            $table->date('tanggal_kembali')->nullable();
            $table->enum('status', ['Dipinjam', 'Dikembalikan'])->default('Dipinjam');
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('peminjamans');
    }
};

```

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('peminjamans', function (Blueprint $table) {
            $table->string('bukti')->nullable()->after('status');
            $table->string('nama_bukti')->nullable()->after('bukti');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('peminjamans', function (Blueprint $table) {
            $table->dropColumn(['bukti', 'nama_bukti']);
        });
    }
};

```

Nama tabel saya tulis eksplisit sebagai peminjamans, karena bentuk jamak otomatis Eloquent untuk kata Peminjaman akan menjadi peminjamen. Model Peminjaman menyimpan daftar status yang sah di konstanta STATUSES, meng-cast kedua tanggal ke format Y-m-d, dan punya accessor buktiUrl yang otomatis ikut di JSON agar JavaScript cukup memakai item.bukti_url.

```php
<?php

namespace Modules\Peminjaman\Models;

use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Modules\Peminjaman\Database\Factories\PeminjamanFactory;

class Peminjaman extends Model
{
    /** @use HasFactory<PeminjamanFactory> */
    use HasFactory;

    /**
     * The table associated with the model.
     *
     * "Peminjaman" ditulis apa adanya, bukan bentuk jamak otomatis "peminjamen".
     *
     * @var string
     */
    protected $table = 'peminjamans';

    /**
     * Status peminjaman yang sah.
     *
     * @var list<string>
     */
    public const STATUSES = ['Dipinjam', 'Dikembalikan'];

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'nama_peminjam',
        'judul_buku',
        'tanggal_pinjam',
        'tanggal_kembali',
        'status',
        'bukti',
        'nama_bukti',
    ];

    /**
     * The accessors to append to the model's array and JSON form.
     *
     * @var list<string>
     */
    protected $appends = [
        'bukti_url',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'tanggal_pinjam' => 'date:Y-m-d',
            'tanggal_kembali' => 'date:Y-m-d',
        ];
```

3.8 Validasi dan Controller Peminjaman

PeminjamanRequest membedakan aturan upload sesuai mode. Saat membuat data baru bukti wajib diunggah, sedangkan saat mengubah data bukti boleh kosong supaya file lama tetap dipakai. Format yang diterima hanya jpg, jpeg, png, dan pdf dengan ukuran maksimal 2 MB. Tanggal kembali boleh kosong, tetapi kalau diisi tidak boleh lebih awal dari tanggal pinjam.

```php
<?php

namespace Modules\Peminjaman\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Modules\Peminjaman\Models\Peminjaman;

class PeminjamanRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * The file is required when creating, and optional when
     * updating (keep the old file).
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $peminjaman = $this->route('peminjaman');

        return [
            'nama_peminjam' => ['required', 'string', 'max:255'],
            'judul_buku' => ['required', 'string', 'max:255'],
            'tanggal_pinjam' => ['required', 'date'],
            'tanggal_kembali' => [
                'nullable',
                'date',
                'after_or_equal:tanggal_pinjam',
            ],
            'status' => ['required', Rule::in(Peminjaman::STATUSES)],
            'bukti' => [
                $peminjaman ? 'nullable' : 'required',
                'file',
                'mimes:jpg,jpeg,png,pdf',
                'max:2048',
            ],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'nama_peminjam.required' => 'Nama peminjam wajib diisi.',
            'judul_buku.required' => 'Judul buku wajib diisi.',
            'tanggal_pinjam.required' => 'Tanggal pinjam wajib diisi.',
            'tanggal_pinjam.date' => 'Tanggal pinjam tidak valid.',
            'tanggal_kembali.after_or_equal' => 'Tanggal kembali tidak boleh '
                .'sebelum tanggal pinjam.',
            'status.required' => 'Status wajib dipilih.',
            'status.in' => 'Status harus Dipinjam atau Dikembalikan.',
            'bukti.required' => 'Bukti peminjaman wajib diunggah.',
```

PeminjamanController tidak punya satu pun method yang mengembalikan redirect. index hanya merender view, sedangkan getData() mengembalikan seluruh data sebagai JSON. Method store dan update menyimpan file lewat store('bukti_peminjaman', 'public') dan mencatat nama aslinya. Pada update, file lama baru dihapus kalau ada file baru yang ikut dikirim. Method destroy menghapus file dari storage lebih dulu sebelum barisnya dihapus.

```php
<?php

namespace Modules\Peminjaman\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Storage;
use Illuminate\View\View;
use Modules\Peminjaman\Http\Requests\PeminjamanRequest;
use Modules\Peminjaman\Models\Peminjaman;

class PeminjamanController extends Controller
{
    /**
     * Display the page. Data is loaded via AJAX.
     */
    public function index(): View
    {
        return view('peminjaman::index');
    }

    /**
     * Return all loans as JSON.
     */
    public function getData(): JsonResponse
    {
        return response()->json(Peminjaman::latest()->get());
    }

    /**
     * Store a newly created loan together with its uploaded proof file.
     */
    public function store(PeminjamanRequest $request): JsonResponse
    {
        $data = $request->safe()->except('bukti');

        if (blank($data['tanggal_kembali'] ?? null)) {
            $data['tanggal_kembali'] = null;
        }

        $file = $request->file('bukti');
        $data['bukti'] = $file->store('bukti_peminjaman', 'public');
        $data['nama_bukti'] = $file->getClientOriginalName();

        Peminjaman::create($data);

        return response()->json(['success' => 'Data peminjaman berhasil disimpan.']);
    }

    /**
     * Return a single loan for the edit form.
     */
    public function edit(Peminjaman $peminjaman): JsonResponse
    {
        return response()->json($peminjaman);
    }

    /**
     * Update the specified loan. The old file is replaced only
     * when a new one is uploaded.
     */
    public function update(
        PeminjamanRequest $request,
```

Route peminjaman ditulis satu per satu karena ada route tambahan peminjaman/data yang mengembalikan JSON. Semua route memakai middleware auth dan role:admin,user, jadi admin dan anggota bisa sama-sama mencatat peminjaman.



```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\Peminjaman\Http\Controllers\PeminjamanController;

Route::middleware(['auth', 'role:admin,user'])->group(function () {
    Route::get('peminjaman', [PeminjamanController::class, 'index'])->name('peminjaman.index');
    Route::get('peminjaman/data', [PeminjamanController::class, 'getData'])->name('peminjaman.data');
    Route::post('peminjaman', [PeminjamanController::class, 'store'])->name('peminjaman.store');
    Route::get('peminjaman/{peminjaman}/edit', [PeminjamanController::class, 'edit'])->name('peminjaman.edit');
    Route::put('peminjaman/{peminjaman}', [PeminjamanController::class, 'update'])->name('peminjaman.update');
    Route::delete('peminjaman/{peminjaman}', [PeminjamanController::class, 'destroy'])->name('peminjaman.destroy');
});

```

3.9 Dashboard dan Navigasi

DashboardController menghitung total buku, jumlah peminjaman berstatus Dipinjam, dan total transaksi. Total user hanya dihitung dan ditampilkan kalau pengguna yang membuka dashboard adalah admin, sehingga anggota tidak melihat angka akun yang tidak relevan untuknya.

```php
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\View\View;
use Modules\Buku\Models\Buku;
use Modules\Peminjaman\Models\Peminjaman;

class DashboardController extends Controller
{
    /**
     * Show the dashboard with record counts per module.
     */
    public function __invoke(Request $request): View
    {
        return view('dashboard', [
            'totalBuku' => Buku::count(),
            'totalDipinjam' => Peminjaman::where('status', 'Dipinjam')->count(),
            'totalPeminjaman' => Peminjaman::count(),
            'totalUsers' => $request->user()->isAdmin() ? User::count() : null,
        ]);
    }
}

```

Layout resources/views/layouts/app.blade.php memuat Bootstrap dan Bootstrap Icons dari CDN. Menu Dashboard, Buku, dan Peminjaman selalu tampil untuk pengguna yang sudah login, sedangkan menu Kelola User dibungkus @if (auth()->user()->isAdmin()). Menu yang disembunyikan hanyalah lapisan tampilan. Penolakan sebenarnya tetap dilakukan middleware role di server.

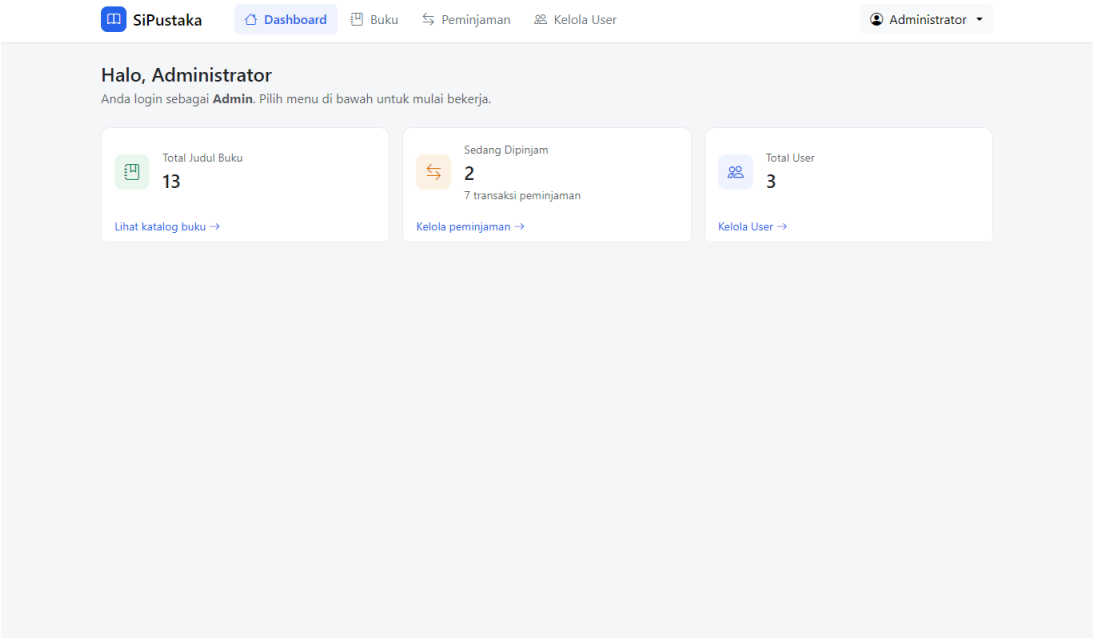
3.10 Menjalankan Aplikasi dan Menguji

Saya mengecek hasilnya dalam dua sesi browser. Sesi pertama memakai akun admin untuk membuka seluruh halaman, dan sesi kedua memakai akun user untuk memastikan menu Kelola User hilang dan alamat /admin/users ditolak dengan halaman 403. Setelah itu saya menguji ulang dari sisi server dengan sesi login yang sama. Hasilnya, anggota yang membuka /admin/users dan /buku/create mendapat status 403, sementara /buku dan /peminjaman tetap 200. Admin mendapat 200 pada /admin/users dan /buku/create. Permintaan ke /peminjaman/data membalas 200 dengan Content-Type application/json sebesar 2.529 byte, dan pengiriman peminjaman tanpa bukti dibalas 422. Lewat browser, tabel peminjaman langsung terisi setelah data diambil tanpa memuat ulang halaman.

4. Hasil dan Dokumentasi Program

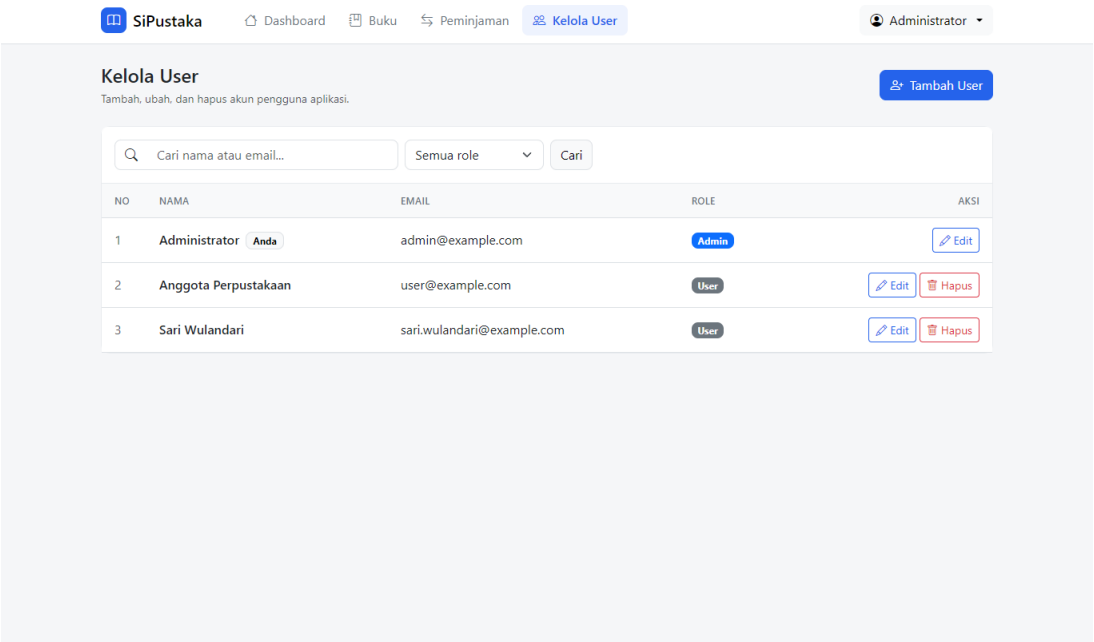
Aplikasi sudah berjalan dan seluruh halaman yang saya dokumentasikan di bawah ini saya buka langsung di browser pada sisi server yang sama. Sesuai rancangan, php artisan route:list --except-vendor menampilkan 25 route. Tiga modul berstatus Enabled pada php artisan module:list, dan migration untuk tabel users dengan kolom role, bukus, peminjamans, serta kolom bukti sudah berstatus Ran.

Halaman login dan register berada di luar modul. Akun yang masuk sebagai admin melihat menu tambahan Kelola User di kanan atas, sedangkan akun anggota hanya melihat Dashboard, Buku, dan Peminjaman.



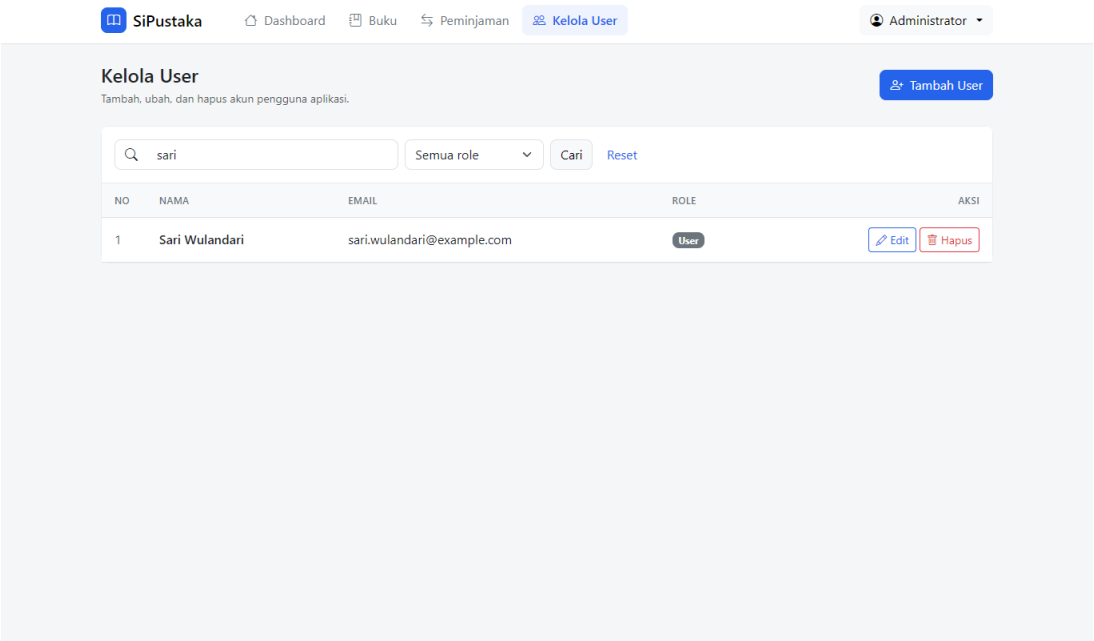
Gambar 1. Dashboard saat login sebagai admin, lengkap dengan total buku, jumlah yang sedang dipinjam, dan total user

Modul UserManagement menampilkan daftar akun beserta role-nya. Baris milik akun yang sedang login diberi badge Anda, sedangkan tombol Hapus disembunyikan di baris itu, sehingga admin tidak bisa memakai tombol tersebut untuk menghapus dirinya sendiri.



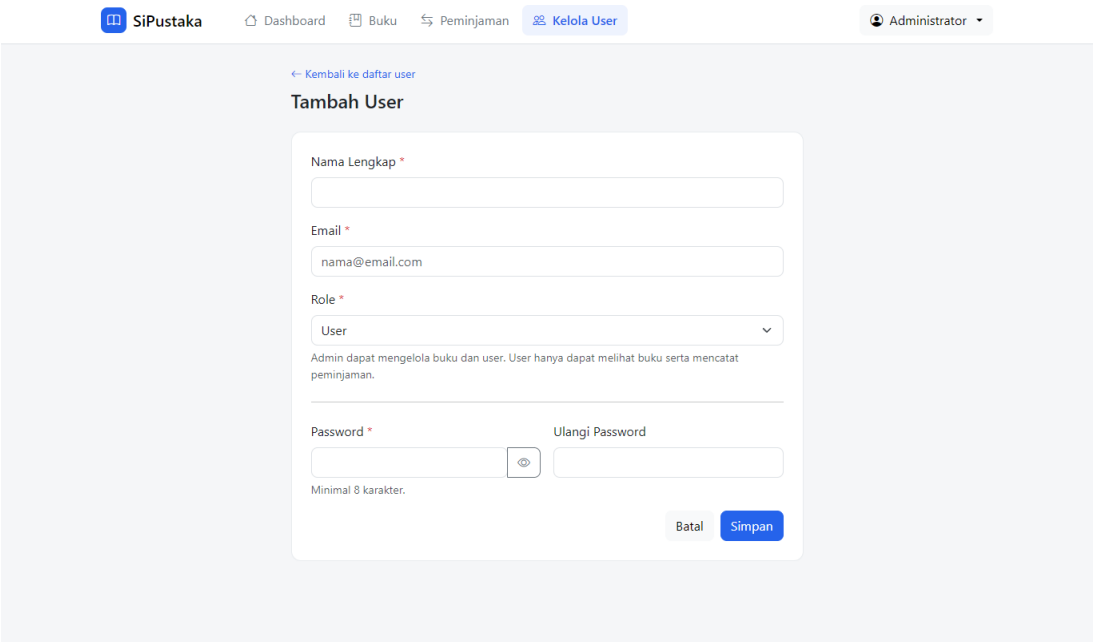
Gambar 2. Daftar akun pada modul UserManagement

Pencarian pada halaman ini memakai satu form GET yang berisi kata kunci dan pilihan role. Tombol Reset muncul hanya ketika salah satu filter sedang aktif.



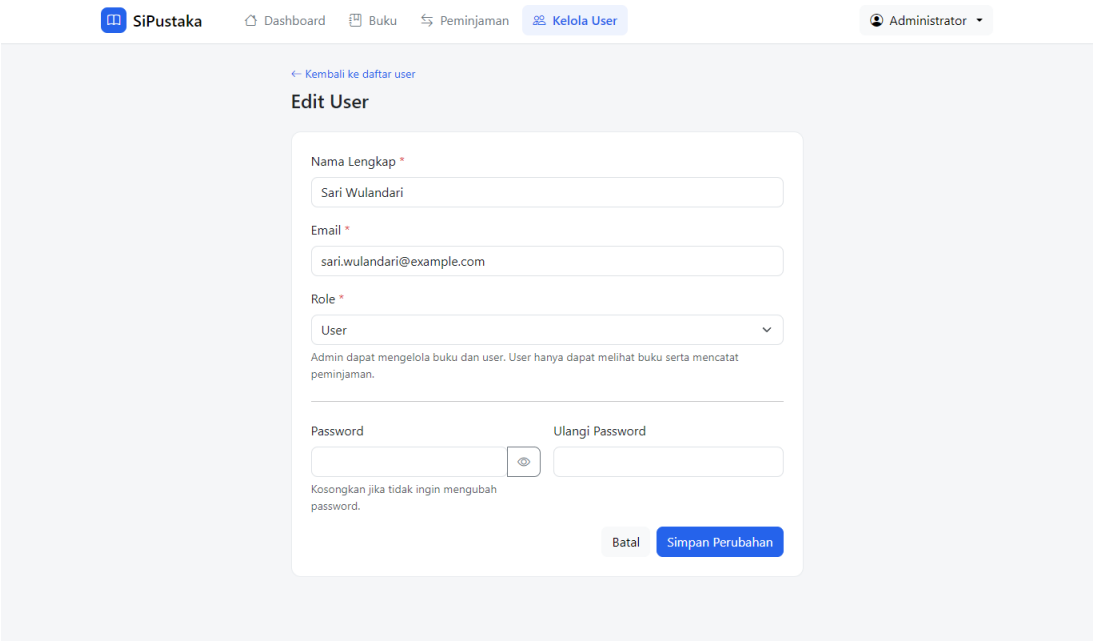
Gambar 3. Hasil pencarian dengan kata kunci sari, menyisakan satu akun

Form tambah dan edit user memakai partial yang sama. Pilihan role berisi Admin dan User, dan pada form edit role akun sendiri hanya boleh tetap Admin.



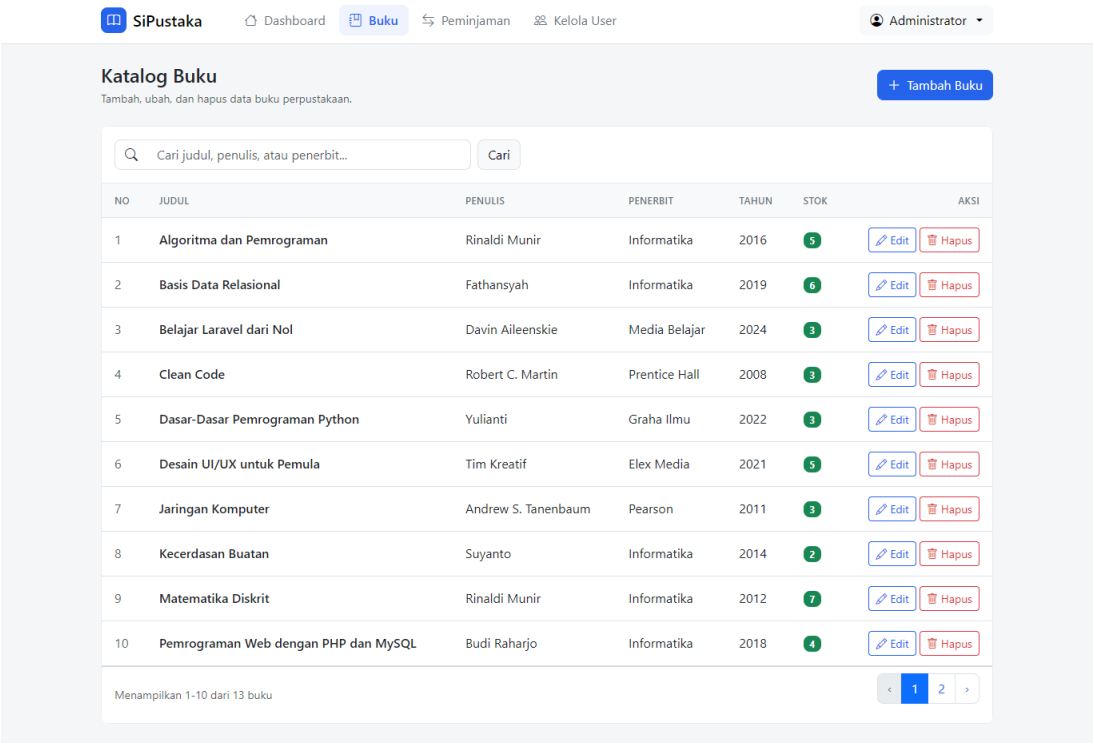
Gambar 4. Form tambah user pada modul UserManagement

Form edit memakai partial yang sama dengan form tambah, jadi tata letaknya identik. Perbedaannya hanya pada data yang sudah terisi dan judul halamannya.



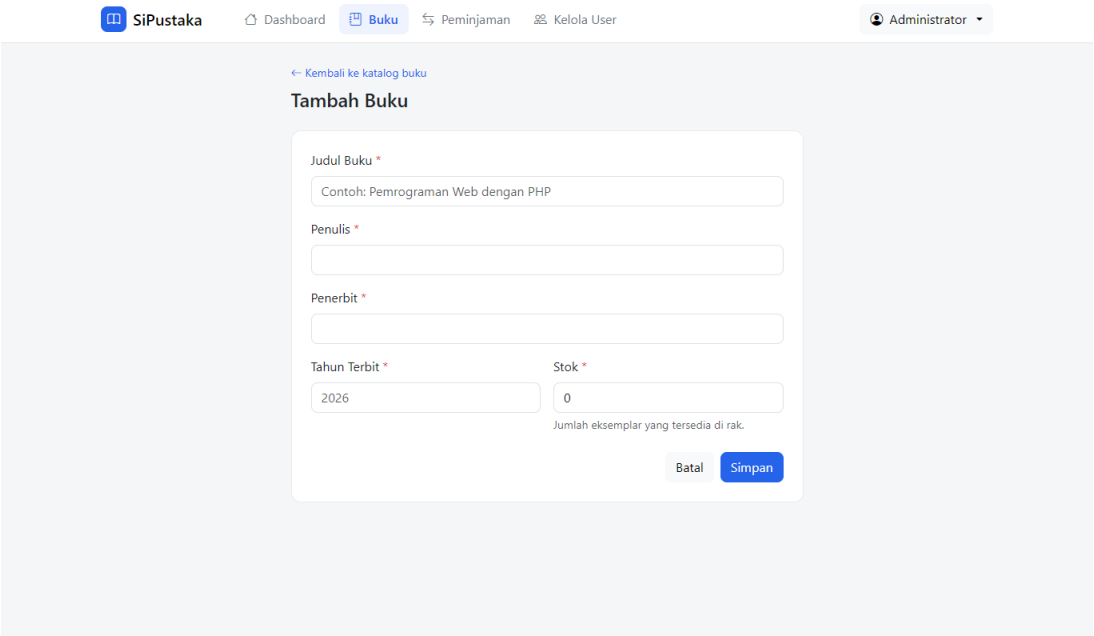
Gambar 5. Form edit akun Sari Wulandari

Katalog menampilkan judul, penulis, penerbit, tahun, dan stok. Akun admin melihat tombol Tambah Buku, Edit, dan Hapus di setiap baris.



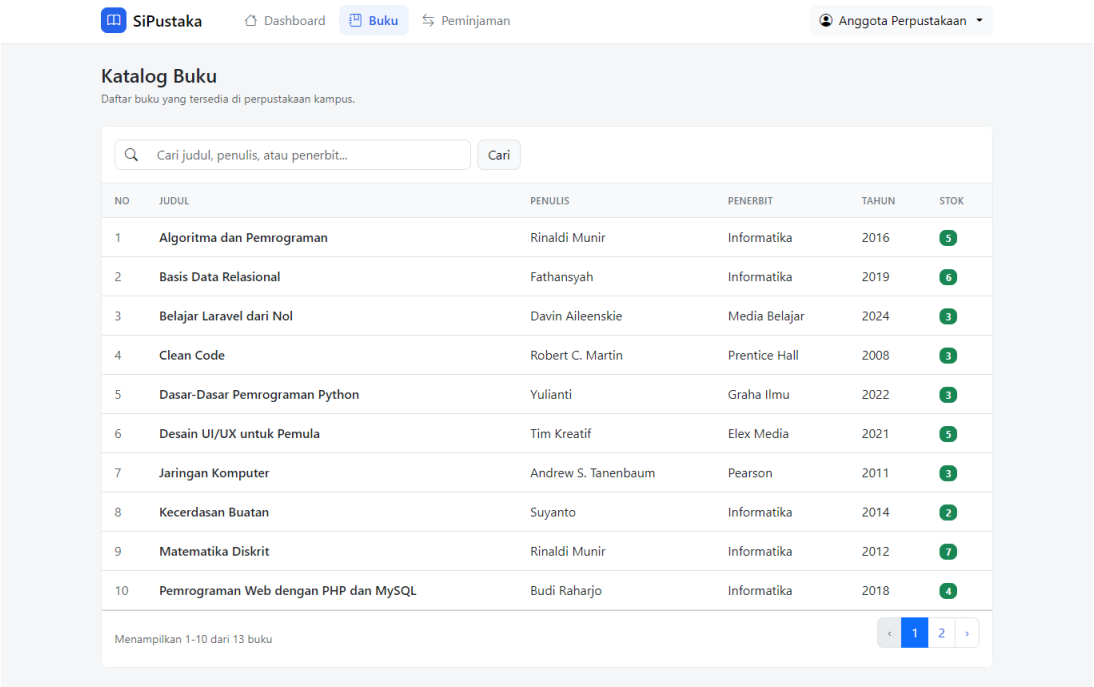
Gambar 6. Katalog buku saat dibuka sebagai admin, dengan tombol aksi

Form tambah buku berisi lima isian, yaitu judul, penulis, penerbit, tahun terbit, dan stok. Tahun terbit divalidasi agar tidak melewati tahun berjalan supaya salah ketik seperti 20206 langsung ditolak.



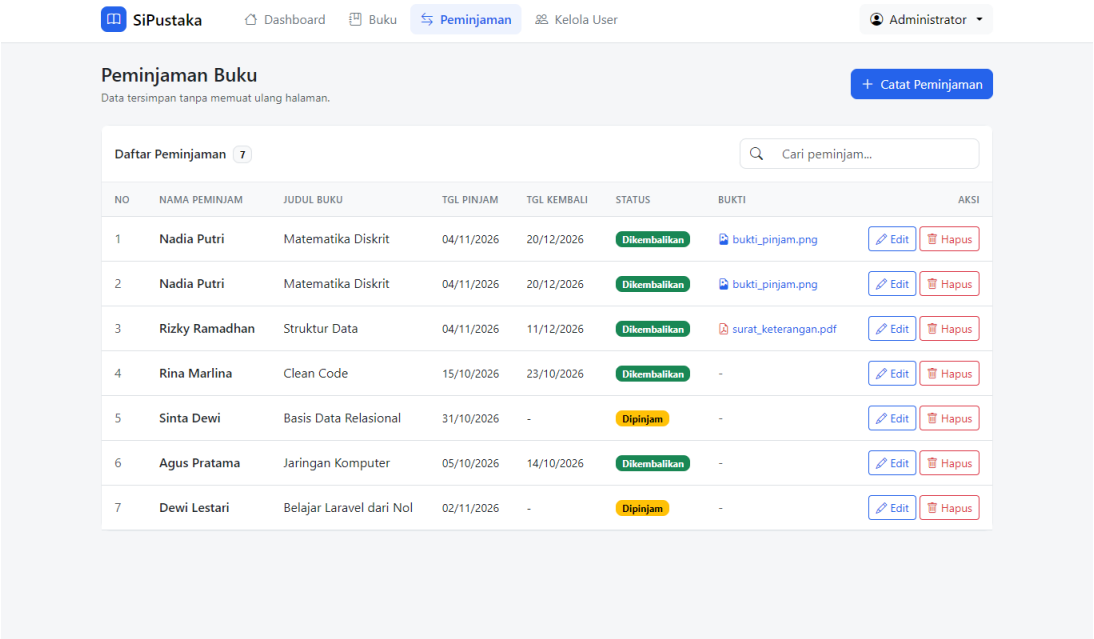
Gambar 7. Form tambah buku

Halaman yang sama saat dibuka sebagai anggota tidak memiliki kolom Aksi maupun tombol Tambah Buku, tetapi isi katalognya tetap lengkap.



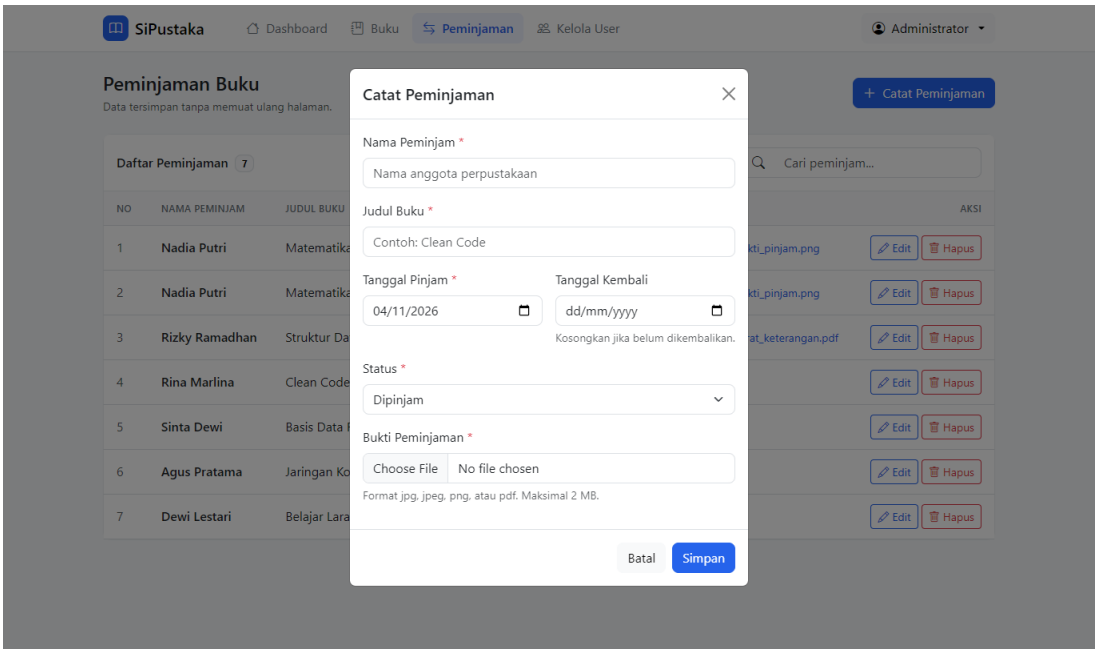
Gambar 8. Katalog buku saat dibuka sebagai anggota, tanpa tombol aksi

Modul Peminjaman memuat datanya lewat AJAX. Tabel di bawah tampil setelah getData() selesai dipanggil, dan jumlah data ditunjukkan di samping judul tabel. Kolom Bukti berisi tautan ke file yang diunggah, dengan ikon berbeda untuk gambar dan PDF.



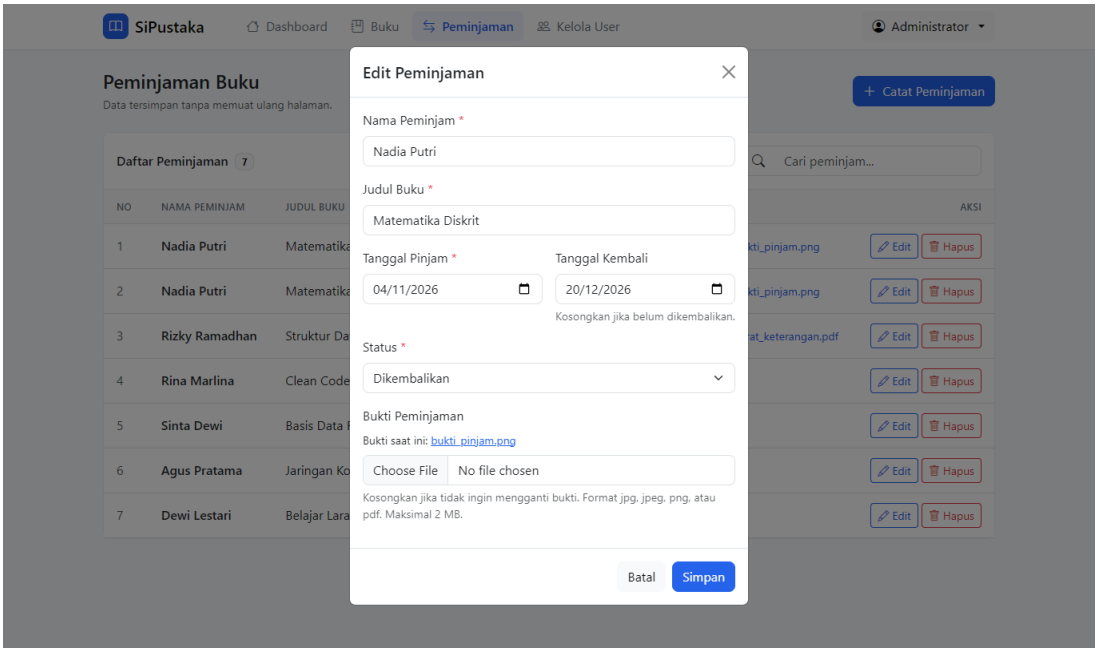
Gambar 9. Daftar peminjaman yang dimuat lewat AJAX

Tombol Catat Peminjaman membuka modal berisi form. Isian yang wajib ditandai bintang, dan input bukti hanya menerima jpg, jpeg, png, atau pdf dengan ukuran maksimal 2 MB.



Gambar 10. Modal catat peminjaman pada modul AJAX

Tombol Edit mengisi form dengan data yang sudah ada lewat route peminjaman.edit, lalu modal yang sama dipakai untuk menyimpan perubahan. Pada mode ini label wajib di input bukti hilang karena file boleh tidak diganti.



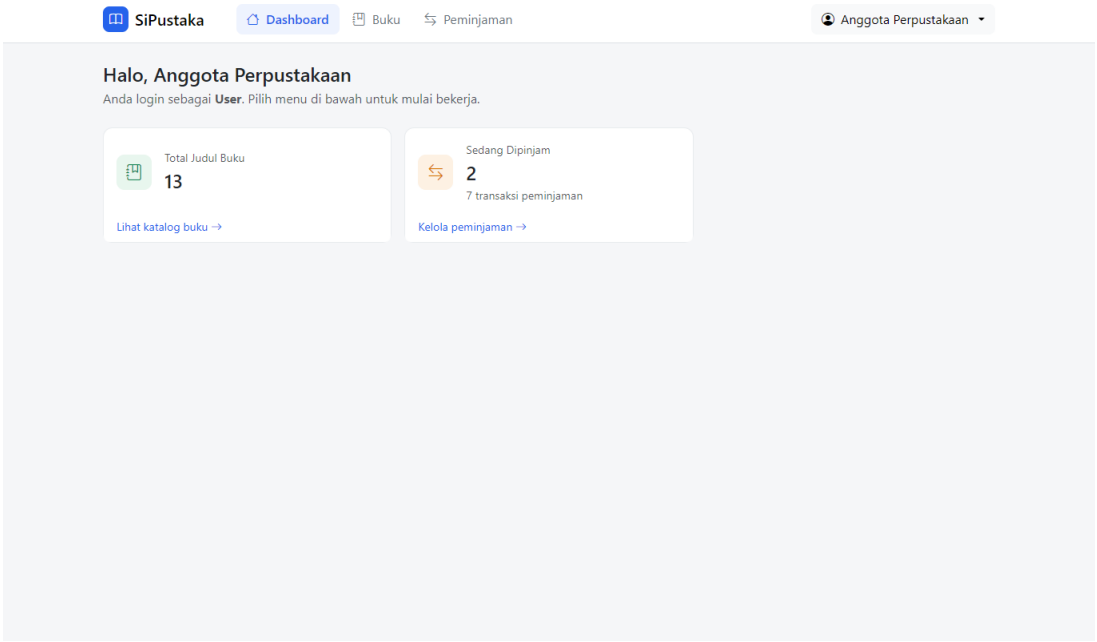
Gambar 11. Modal edit peminjaman yang terisi data lama beserta tautan bukti saat ini

Untuk menguji otorisasi, saya login dengan akun user@example.com lalu membuka /admin/users langsung dari address bar. Middleware role:admin menghentikan request sebelum controller dijalankan, sehingga yang tampil adalah halaman 403 dengan pesan dari middleware.



Gambar 12. Halaman 403 saat akun anggota membuka alamat khusus admin

Dashboard untuk anggota tidak menampilkan kartu Total User. Menu Kelola User juga tidak ada di navigasi, hanya Dashboard, Buku, dan Peminjaman.



Gambar 13. Dashboard saat login sebagai anggota perpustakaan

5. Analisis dan Pembahasan

1. Pembagian tanggung jawab antar modul.

Setiap modul di project ini berdiri sendiri tetapi berjalan dalam satu aplikasi. Buku menyimpan katalog, UserManagement menyimpan daftar akun, dan Peminjaman menyimpan transaksi. Yang saya pisahkan adalah kode fiturnya: controller, request, model, migration, dan view milik satu fitur ada dalam satu folder Modules. Route admin/users diberi prefix dan awalan nama admin., sedangkan route buku dan peminjaman memakai nama yang sederhana karena tidak bergantung pada role tertentu. Namespace view seperti buku::index, peminjaman::index, dan usermanagement::index memastikan tiga view dengan nama sama tidak saling menimpa.

2. Satu middleware untuk tiga pola akses.

Ada tiga pola pemakaian middleware role:admin di lima route perubahan buku, role:admin di seluruh UserManagement, serta role:admin,user di index buku, semua route peminjaman, dan dashboard. Daripada menulis pemeriksaan role di setiap controller, saya memusatkannya di EnsureUserHasRole. Konsekuensinya jelas: pengecekan terjadi sebelum controller dipanggil, dan request yang ditolak tidak pernah menyentuh database. Enum UserRole membuat daftar role yang diterima konsisten, karena string yang tidak cocok akan menghasilkan tryFrom() bernilai null dan otomatis gagal.

3. Autentikasi di luar modul, otorisasi di dalam route.

LoginController dan RegisterController berada di app/Http/Controllers/Auth, bukan di dalam Modules. Bentuknya memang sengaja begitu: autentikasi menjawab siapa pengguna, dan itu dibutuhkan sebelum modul mana pun tahu harus menampilkan apa. RegisterRequest tidak menerima role dari pengguna, controller mengisi UserRole::User sendiri. Setelah status pengguna diketahui, modul baru memakai informasi itu lewat middleware dan helper isAdmin().

4. Dua gaya CRUD yang dipakai bersamaan.

UserManagement dan Buku memakai gaya sinkron. Form dikirim ke controller, controller melakukan redirect dengan pesan session, lalu halaman dimuat ulang. Cara ini cocok karena frekuensi pemakaiannya tidak tinggi dan validasi bisa langsung memakai @error bawaan Blade. Peminjaman memakai AJAX karena tabelnya dianggap sebagai daftar transaksi yang sering berubah. Semua method tulis di controller mengembalikan JSON, dan tabel direnderer ulang dari data yang baru diambil tanpa memuat dokumen baru. Validasi 422 dari laravel dipetakan ke elemen invalid-feedback di bawah tiap input.

5. Unggah berkas lewat AJAX.

Form peminjaman dikirim sebagai FormData, bukan serialize(), karena ada file di dalamnya. Dengan processData dan contentType disetel false, jQuery tidak mengubah FormData menjadi query string, sehingga file ikut terkirim. Saat mengubah data, _method PUT ditambahkan ke FormData supaya Laravel tetap membaca route update. File disimpan di storage/app/public/bukti_peminjaman dengan nama acak, sedangkan database hanya menyimpan path dan nama aslinya. php artisan storage:link sudah saya jalankan, jadi tautan public/storage bisa membuka file tersebut dari tabel.

6. Keamanan tampilan dan server.

Menu Kelola User, tombol Tambah Buku, dan kolom Aksi disembunyikan dengan @if (auth()->user()->isAdmin()). Kondisi di view tidak bisa diandalkan sebagai keamanan, karena pengguna tetap bisa mengetik alamat atau mengirim request manual. Karena itu route tetap memakai middleware, dan Form Request seperti StoreBukuRequest juga memeriksa isAdmin() pada authorize(). Data yang dirender ke tabel peminjaman melewati escapeHtml() sebelum dimasukkan ke template string, supaya nama peminjam tidak bisa dijadikan kode HTML.

7. Kendala dan penyelesaian.

Kendala yang saya catat diambil dari storage/logs/laravel.log. Galat pertama adalah SQLSTATE[42S02] Table 'db_sipustaka.peminjamen' doesn't exist. Penyebabnya nama tabel yang ditebak Eloquent, karena kata Peminjaman dianggap berbahasa Inggris dan bentuk jamaknya menjadi peminjamen. Saya menambahkan protected $table = 'peminjamans' pada model dan tabelnya dikenali lagi.

Galat kedua muncul saat saya menjalankan perintah artisan, yaitu The "--json" option does not exist dan Command "module:clear-cache" is not defined. Dua perintah itu memang tidak tersedia pada versi Laravel 12.71.0 dan nwidart/laravel-modules v12.0.5 yang saya pakai, jadi saya memeriksa modul dengan php artisan module:list dan menghapus cache lewat php artisan optimize:clear. Galat ketiga berupa MissingInputException dengan pesan Aborted, yang muncul ketika sebuah perintah artisan interaktif saya hentikan dengan Ctrl+C.

8. Perbedaan dengan modul.

Modul contoh memakai skema pendaftaran mahasiswa dengan tabel daftars, sedangkan project ini memakai kasus perpustakaan dengan tabel bukus dan peminjamans. Modul juga membuat satu modul dengan CRUD penuh AJAX, sementara di sini CRUD AJAX dipakai pada Peminjaman dan CRUD sinkron dipakai pada dua modul lain. Pada modul, halaman user dibuat dengan Breeze; di project ini autentikasi ditulis sendiri dan tampilan memakai Bootstrap dari CDN. Tangkapan layar di Bab 4 saya ambil melalui browser headless pada alamat 127.0.0.1:8018 yang dijalankan dengan php artisan serve.

6. Kesimpulan

Aplikasi perpustakaan SiPustaka berhasil dipecah menjadi tiga modul dengan paket nwidart/laravel-modules. Modul Buku, Peminjaman, dan UserManagement masing-masing punya route, controller, request, model, migration, dan view sendiri, dan ketiganya terdaftar Enabled. Autentikasi register dan login tetap berada di luar modul, dengan akun baru selalu berrole user.

Otorisasi berjalan sesuai rancangan. Admin bisa membuka seluruh halaman, mengelola akun, dan mengubah data buku, sedangkan anggota hanya melihat katalog dan mencatat peminjaman. Akun anggota yang membuka /admin/users ditolak dengan halaman 403 dari middleware, dan menu Kelola User tidak muncul di navigasinya. Modul Peminjaman

menyimpan, mengubah, dan menghapus data lewat AJAX tanpa memuat ulang halaman, termasuk mengganti bukti dengan FormData dan menampilkan pesan validasi per input.

7. Link Repository Github

https://github.com/davinaileenskie1-hue/PABW-Semester-3